In [ ]:
```
import marimo as mo
```

Можно ли смоделировать экономическое поведение общества биологическим нейроном?

Цель исследования

Мне захотелось из любопытства взять математическую модель биологического нейрона и попробовать с её помощью описать реальный процесс. Теперь вместо солнечных пятен мы исследуем **потребительские настроения**: насколько оптимистично или пессимистично люди оценивают экономику и собственное финансовое положение.

Это не попытка утверждать, что общество буквально работает как мозг. Мы проверяем более узкую гипотезу: **может ли динамика модели ФицХью—Нагумо (FHN) быть полезным способом представить и прогнозировать изменения общественного экономического настроения?**

Почему именно этот показатель?

Индекс потребительских настроений Университета Мичигана (UMCSENT) — ежемесячный опросный индикатор в США, доступный с 1952 года. Он отражает ответы потребителей о финансовом положении и экономических ожиданиях. Это измеримый общественный показатель с достаточно длинной историей, в которой встречаются подъёмы, спады и кризисы.

Источник ряда: FRED, серия `UMCSENT`, первичный источник — Surveys of Consumers, University of Michigan. См. страницу ряда.

> Ограничение интерпретации: индекс настроений — не прямое измерение расходов домохозяйств и не мнение каждого человека. Один ряд не позволяет заключить, что мы моделируем «всё общество».

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import differential_evolution
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import StandardScaler
```

1. Получаем данные и задаём честный эксперимент

Ряд загружается из общедоступного CSV-экспорта FRED. Для запуска нужен интернет. Мы используем месячные наблюдения, а не годовые: так у нас больше точек и можно исследовать реакцию на изменения в течение нескольких месяцев.

Разделение хронологическое:

- первые 70% — обучение;
- следующие 10% — валидация, на которой подбираются параметры одиночного нейрона;
- последние 20% — тест, который открываем только для итогового сравнения.

Почему так? Случайно перемешивать временной ряд нельзя: тогда модель может косвенно использовать информацию из будущего. Масштабирование также обучается только на тренировочной части.

In [ ]:
```
url = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UMCSENT"
raw = pd.read_csv(url)
raw.columns = ["date", "value"]
raw["date"] = pd.to_datetime(raw["date"], errors="coerce")
raw["value"] = pd.to_numeric(raw["value"], errors="coerce")
raw = raw.dropna().sort_values("date").reset_index(drop=True)
dates = raw["date"].to_numpy()
y = raw["value"].to_numpy(dtype=float)
n = len(y)
train_end = int(n * 0.70)
val_end = int(n * 0.80)
scaler = StandardScaler().fit(y[:train_end, None])
z = scaler.transform(y[:, None]).ravel()
data_summary = pd.DataFrame({
    "Показатель": ["Источник", "Частота", "Период", "Наблюдений",
"Обучение", "Валидация", "Тест"],
    "Значение": ["FRED: UMCSENT", "Ежемесячно", f"
{pd.Timestamp(dates[0]).date()} — {pd.Timestamp(dates[-1]).date()}", n,
                 f"{pd.Timestamp(dates[0]).date()} —
{pd.Timestamp(dates[train_end-1]).date()} ({train_end})",
                 f"{pd.Timestamp(dates[train_end]).date()} —
{pd.Timestamp(dates[val_end-1]).date()} ({val_end-train_end})",
                 f"{pd.Timestamp(dates[val_end]).date()} —
{pd.Timestamp(dates[-1]).date()} ({n-val_end})"]
})
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(dates[:train_end], y[:train_end], label="Обучение")
ax.plot(dates[train_end:val_end], y[train_end:val_end], label="Валидация")
ax.plot(dates[val_end:], y[val_end:], label="Тест")
ax.axvline(dates[train_end], color="black", linestyle="--", alpha=.6)
```

```
ax.axvline(dates[val_end], color="black", linestyle=":", alpha=.6)
ax.set(title="Индекс потребительских настроений США (UMCSENT)",
xlabel="Дата", ylabel="Индекс")
ax.grid(alpha=.25)
ax.legend(frameon=False)
fig.tight_layout()
mo.vstack([mo.ui.table(data_summary), fig])
```

Search...

Показатель str	Значение object
Источник	FRED: UMCSENT
Частота	Ежемесячно
Период	1952-11-01 — 2026-08-01
Наблюдений	676
Обучение	1952-11-01 — 2009-09-01 (473)
Валидация	2009-10-01 — 2015-04-01 (67)
Тест	2015-05-01 — 2026-08-01 (136)

7 rows, 2 columns *No selection*



2. Что означает FHN в этой задаче?

Упрощённая модель ФицХью—Нагумо имеет две переменные состояния:

$$\frac{dv}{dt} = v - \frac{v^3}{3} - w + I(t), \qquad \frac{dw}{dt} = \varepsilon(v + a - bw).$$

`v` — быстро меняющееся состояние нейрона, `w` — более медленная переменная восстановления. В биологии это упрощённое описание возбудимой клетки.

Для эксперимента будем подавать нормированное значение индекса в качестве внешнего воздействия `I(t)`. Сами `v` и `w` не являются настоящими экономическими переменными. Это скрытые состояния математического преобразователя, из которых модель пытается предсказать следующий месяц.

Параметры `a`, `b`, `epsilon` и сила входного сигнала меняют динамику нейрона. Для одного нейрона мы подберём их по валидационной ошибке. В ансамбле

используем набор нейронов с разными параметрами и обучим линейный выходной слой по их состояниям.

In [ ]:
```
def simulate_one(inputs, params, dt=0.05):
    a, b, eps, gain, bias = params
    v, w = -1.0, -0.5
    states = np.zeros((len(inputs), 2), dtype=float)
    for i, u in enumerate(inputs):
        current = bias + gain * float(u)
        dv = v - v**3 / 3.0 - w + current
        dw = eps * (v + a - b * w)
        v += dt * dv
        w += dt * dw
        v = float(np.clip(v, -5, 5))
        w = float(np.clip(w, -5, 5))
        states[i] = (v, w)
    return states

def make_supervised(states, z):
    # Состояние после входа месяца t предсказывает индекс месяца t+1.
    X = np.column_stack([states[:-1], z[:-1]])
    target = z[1:]
    return X, target
```

3. Базовые методы — обязательная контрольная группа

Прежде чем доверять сложной модели, сравним её с двумя простыми прогнозами:

1. **Persistence:** следующий месяц равен текущему. Это сильный ориентир для ряда, который часто меняется постепенно.
2. **Ridge с лагами:** линейная регрессия использует последние 3 значения индекса. Она показывает, может ли простая модель временной инерции справиться не хуже FHN.

Если FHN не превосходит эти ориентиры на тесте, это означает, что в текущей постановке его дополнительная сложность не дала измеримого преимущества.

In [ ]:
```
# Индекс i в supervised-выборке соответствует прогнозу z[i+1].
target_indices = np.arange(1, len(z))
X_lag = np.column_stack([z[:-1], np.roll(z, 1)[:-1], np.roll(z, 2)[:-1]])
# Первые строки с искусственно циклическими лагами исключаем из обучения/
оценки.
X_lag = np.column_stack([z[2:-1], z[1:-2], z[:-3]])
y_lag = z[3:]
idx_lag = np.arange(3, len(z))
m_train = idx_lag < train_end
m_val = (idx_lag >= train_end) & (idx_lag < val_end)
m_test = idx_lag >= val_end
```

```
lag_model = Ridge(alpha=10.0).fit(X_lag[m_train], y_lag[m_train])
lag_pred = lag_model.predict(X_lag[m_test])
actual_test = z[idx_lag[m_test]]
persistence_pred = z[idx_lag[m_test] - 1]
baseline_metrics = {
    "Persistence": (float(np.sqrt(mean_squared_error(actual_test,
persistence_pred))), float(mean_absolute_error(actual_test,
persistence_pred))),
    "Ridge: 3 лага": (float(np.sqrt(mean_squared_error(actual_test,
lag_pred))), float(mean_absolute_error(actual_test, lag_pred)))
}
```

4. Эксперимент A — один настроенный FHN-нейрон

У одиночного нейрона есть немного параметров, поэтому можно систематически искать их комбинацию. Используем `differential_evolution` — метод глобальной оптимизации, который исследует диапазоны параметров без необходимости вычислять градиент.

Критерий подбора — ошибка на валидационном участке, а не на тесте. После подбора выходной Ridge-слой обучается на тренировочном периоде. Тестовые значения остаются нетронутыми до сравнения.

In [ ]:
```
def one_model_validation_loss(params):
    states = simulate_one(z[:val_end], params)
    X, target = make_supervised(states, z[:val_end])
    # X[k] predicts z[k+1]. Validation targets begin at train_end.
    train_mask = np.arange(1, val_end) < train_end
    val_mask = (np.arange(1, val_end) >= train_end)
    if train_mask.sum() < 20 or val_mask.sum() < 5:
        return 1e6
    readout = Ridge(alpha=10.0).fit(X[train_mask], target[train_mask])
    pred = readout.predict(X[val_mask])
    return float(np.sqrt(mean_squared_error(target[val_mask], pred)))

bounds = [(-1.5, 1.5), (0.2, 2.0), (0.005, 0.15), (0.0, 2.5), (-1.5, 1.5)]
opt_result = differential_evolution(one_model_validation_loss, bounds,
seed=42, maxiter=18, popsize=7, polish=False, updating="immediate")
best_params = opt_result.x
single_states = simulate_one(z[:], best_params)
X_single, y_single = make_supervised(single_states, z)
single_train_mask = np.arange(1, len(z)) < train_end
single_model = Ridge(alpha=10.0).fit(X_single[single_train_mask],
y_single[single_train_mask])
single_pred_all = single_model.predict(X_single)
single_target_indices = np.arange(1, len(z))
single_test_mask = single_target_indices >= val_end
single_pred = single_pred_all[single_test_mask]
single_actual = z[single_target_indices[single_test_mask]]
single_rmse = float(np.sqrt(mean_squared_error(single_actual,
single_pred)))
```

```
single_mae = float(np.mean(np.abs(single_actual - single_pred)))
params_table = __import__("pandas").DataFrame({"Параметр": ["a", "b",
"epsilon", "gain", "bias"], "Подобранное значение": best_params})
```

In [ ]:
```
mo.vstack([mo.md(f"**Лучшая валидационная RMSE:** {opt_result.fun:.3f} (в
нормированном масштабе)."), mo.ui.table(params_table)])
```

Лучшая валидационная RMSE: 0.297 (в нормированном масштабе).

Search...

Параметр str	Подобранное значение float64
a	-1,46789656124165460
b	1,71809214870536530
epsilon	0,12722538452480525
gain	2,44366596578696440
bias	0,10132917695138677

5 rows, 2 columns — *No selection*

5. Эксперимент B — ансамбль из 48 нейронов

Почему нужен ансамбль? Разные нейроны могут реагировать на один и тот же вход по-разному: один даёт быстрый отклик, другой — более медленный, третий — сильнее реагирует на большие изменения. Линейный выходной слой может комбинировать эти состояния.

В этом первом ансамблевом варианте параметры нейронов выбираются случайно в разумных диапазонах и затем фиксируются. Обучается только Ridge-readout. Это важно явно отметить: **мы не оптимизируем каждый нейрон ансамбля**, поэтому сравнение проверяет, помогает ли разнообразие случайных динамик при фиксированном способе обучения, а не является поиском лучшего возможного ансамбля.

In [ ]:
```
rng = np.random.default_rng(123)
n_neurons = 48
ensemble_params = np.column_stack([
    rng.uniform(-1.0, 1.0, n_neurons),
    rng.uniform(0.4, 1.6, n_neurons),
    rng.uniform(0.01, 0.12, n_neurons),
    rng.uniform(0.2, 1.8, n_neurons),
    rng.uniform(-0.5, 0.5, n_neurons),
])
ensemble_features = []
for p in ensemble_params:
    st = simulate_one(z, p)
    ensemble_features.append(np.column_stack([st, z]))
X_ensemble = np.concatenate(ensemble_features, axis=1)[:-1]
y_ensemble = z[1:]
ens_train_mask = np.arange(1, len(z)) < train_end
ens_model = Ridge(alpha=30.0).fit(X_ensemble[ens_train_mask],
y_ensemble[ens_train_mask])
ens_pred_all = ens_model.predict(X_ensemble)
```

```
ens_target_indices = np.arange(1, len(z))
ens_test_mask = ens_target_indices >= val_end
ensemble_pred = ens_pred_all[ens_test_mask]
ensemble_actual = z[ens_target_indices[ens_test_mask]]
ensemble_rmse = float(np.sqrt(mean_squared_error(ensemble_actual,
ensemble_pred)))
ensemble_mae = float(mean_absolute_error(ensemble_actual, ensemble_pred))
```

6. Итоговое сравнение на отложенном тесте

Все метрики считаются в нормированном масштабе, чтобы обучение было устойчивым. Для графика также вернём прогнозы в исходную шкалу индекса. Нормированный RMSE удобен для сравнения методов на одном и том же ряде; он не выражен в пунктах исходного индекса.

Важное уточнение: в тестовом режиме здесь делается **прогноз на один месяц вперёд с использованием фактически наблюдённых прошлых значений**. Это не многомесячный прогноз без обновления данных.

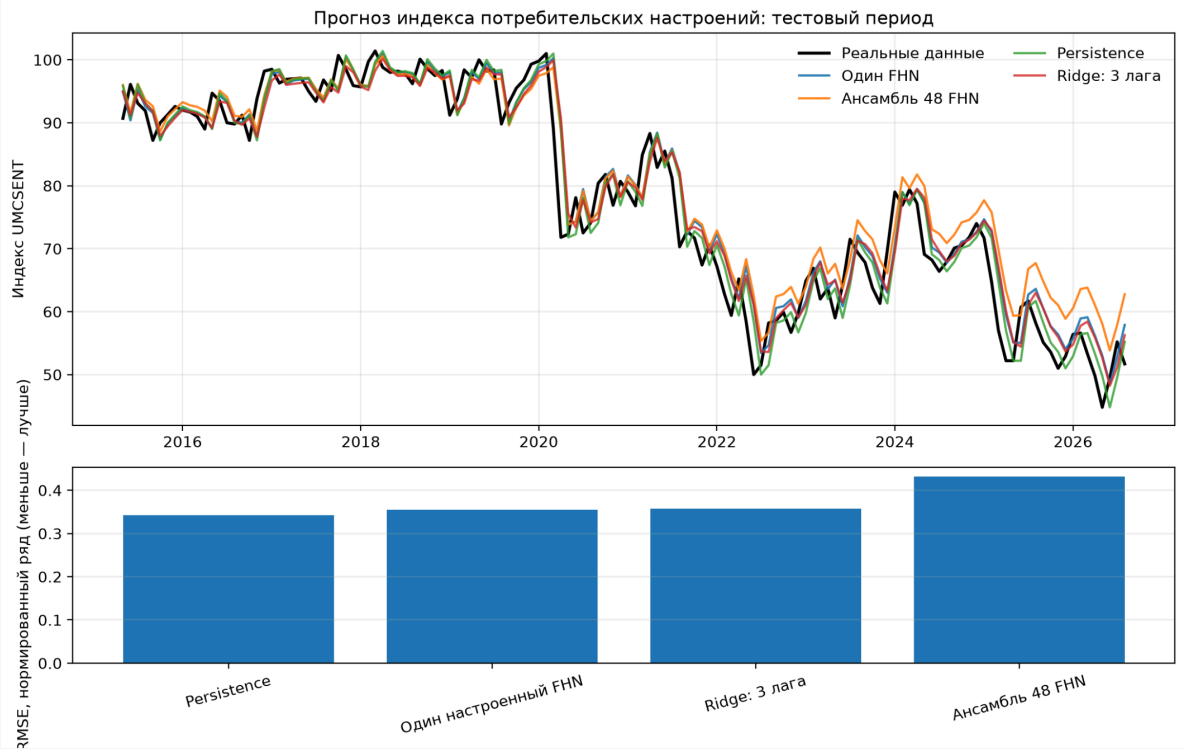
In [ ]:
```
results = pd.DataFrame([
    {"Модель": "Persistence", "RMSE (z)": baseline_metrics["Persistence"]
[0], "MAE (z)": baseline_metrics["Persistence"][1]},
    {"Модель": "Ridge: 3 лага", "RMSE (z)": baseline_metrics["Ridge: 3
лага"][0], "MAE (z)": baseline_metrics["Ridge: 3 лага"][1]},
    {"Модель": "Один настроенный FHN", "RMSE (z)": single_rmse, "MAE (z)":
single_mae},
    {"Модель": "Ансамбль 48 FHN", "RMSE (z)": ensemble_rmse, "MAE (z)":
ensemble_mae},
]).sort_values("RMSE (z)").reset_index(drop=True)
fig_compare, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 7), gridspec_kw=
{"height_ratios": [2, 1]})
test_dates = dates[val_end:]
actual_original = scaler.inverse_transform(z[val_end:, None]).ravel()
ax1.plot(test_dates, actual_original, label="Реальные данные",
color="black", linewidth=2)
single_dates = dates[single_target_indices[single_test_mask]]
single_orig = scaler.inverse_transform(single_pred[:, None]).ravel()
ensemble_orig = scaler.inverse_transform(ensemble_pred[:, None]).ravel()
persistence_orig = scaler.inverse_transform(persistence_pred[:,
None]).ravel()
ridge_orig = scaler.inverse_transform(lag_pred[:, None]).ravel()
ax1.plot(single_dates, single_orig, label="Один FHN", alpha=.9)
ax1.plot(single_dates, ensemble_orig, label="Ансамбль 48 FHN", alpha=.9)
ax1.plot(single_dates, persistence_orig, label="Persistence", alpha=.8)
ax1.plot(dates[idx_lag[m_test]], ridge_orig, label="Ridge: 3 лага",
alpha=.8)
ax1.set_title("Прогноз индекса потребительских настроений: тестовый
период")
ax1.set_ylabel("Индекс UMCSENT")
ax1.grid(alpha=.25)
ax1.legend(frameon=False, ncol=2)
ax2.bar(results["Модель"], results["RMSE (z)"])
```

```
ax2.set_ylabel("RMSE, нормированный ряд (меньше — лучше)")
ax2.tick_params(axis="x", rotation=15)
ax2.grid(axis="y", alpha=.25)
fig_compare.tight_layout()
mo.vstack([mo.ui.table(results.round(4)), fig_compare])
```

Search...

Модель	RMSE (z)	MAE (z)
	str	float64	float64
Persistence	0,3425	0,2650
Один настроенный FHN	0,3548	0,2700
Ridge: 3 лага	0,3577	0,2693
Ансамбль 48 FHN	0,4318	0,3347

4 rows, 3 columns *No selection*



In [ ]:

```
winner = results.iloc[0]
fhn_rows = results[results["Модель"].isin(["Один настроенный FHN",
"Ансамбль 48 FHN"])]
best_fhn = fhn_rows.sort_values("RMSE (z)").iloc[0]
mo.md(f'''## 7. Как интерпретировать результат

**Лучший метод в этом запуске по RMSE:** {winner['Модель']} (RMSE =
{winner['RMSE (z)']:.3f}).

**Лучший из двух FHN-вариантов:** {best_fhn['Модель']} (RMSE =
{best_fhn['RMSE (z)']:.3f}).

Сравнение одиночного нейрона и ансамбля отвечает на вопрос, помогает ли
разнообразие динамик в этой конкретной постановке. Если ансамбль лучше
одиночного FHN, это поддерживает гипотезу о пользе разнообразных состояний;
если хуже — случайно выбранные динамики могли не соответствовать структуре
ряда. Ни один исход сам по себе не доказывает универсальность метода.
```

```
### Почему результат нужно считать предварительным

- Потребительские настроения зависят от инфляции, рынка труда, политики, новостей и других факторов, которые в этой версии отдельно не моделируются.
- Параметры одиночного FHN подбираются по одному историческому валидационному отрезку. Это может привести к переобучению на конкретный период.
- Ансамбль пока использует случайно выбранные параметры; его параметры не оптимизируются индивидуально.
- Результат нужно проверить на других временных разбиениях и, в идеале, на другом индикаторе или стране.

### Что было бы хорошим следующим экспериментом?

1. Повторить проверку на нескольких последовательных тестовых окнах (walk-forward validation).
2. Добавить инфляцию и безработицу как внешние входы, не заглядывая в будущие значения.
3. Сравнить случайный ансамбль с ансамблем, параметры которого тоже оптимизируются.
4. Проверить, устойчив ли вывод при изменении числа нейронов и случайного seed.

**Вывод:** мы не заранее предполагаем, что биологический нейрон способен прогнозировать экономику. Мы ставим проверяемый эксперимент и требуем, чтобы он превзошёл простые методы на данных, которые не участвовали в настройке.''')
```

7. Как интерпретировать результат

Лучший метод в этом запуске по RMSE: Persistence (RMSE = 0.343). **Лучший из двух FHN-вариантов:** Один настроенный FHN (RMSE = 0.355). Сравнение одиночного нейрона и ансамбля отвечает на вопрос, помогает ли разнообразие динамик в этой конкретной постановке. Если ансамбль лучше одиночного FHN, это поддерживает гипотезу о пользе разнообразных состояний; если хуже — случайно выбранные динамики могли не соответствовать структуре ряда. Ни один исход сам по себе не доказывает универсальность метода.

Почему результат нужно считать предварительным

- Потребительские настроения зависят от инфляции, рынка труда, политики, новостей и других факторов, которые в этой версии отдельно не моделируются.
- Параметры одиночного FHN подбираются по одному историческому валидационному отрезку. Это может привести к переобучению на конкретный период.
- Ансамбль пока использует случайно выбранные параметры; его параметры не оптимизируются индивидуально.
- Результат нужно проверить на других временных разбиениях и, в идеале, на другом индикаторе или стране.

Что было бы хорошим следующим экспериментом?

1. Повторить проверку на нескольких последовательных тестовых окнах (walk-forward validation).
2. Добавить инфляцию и безработицу как внешние входы, не заглядывая в будущие значения.
3. Сравнить случайный ансамбль с ансамблем, параметры которого тоже оптимизируются.
4. Проверить, устойчив ли вывод при изменении числа нейронов и случайного seed.

Вывод: мы не заранее предполагаем, что биологический нейрон способен прогнозировать экономику. Мы ставим проверяемый эксперимент и требуем, чтобы он превзошёл простые методы на данных, которые не участвовали в настройке.